

Reinforcement Learning: Basic Definitions (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Summary of today's class

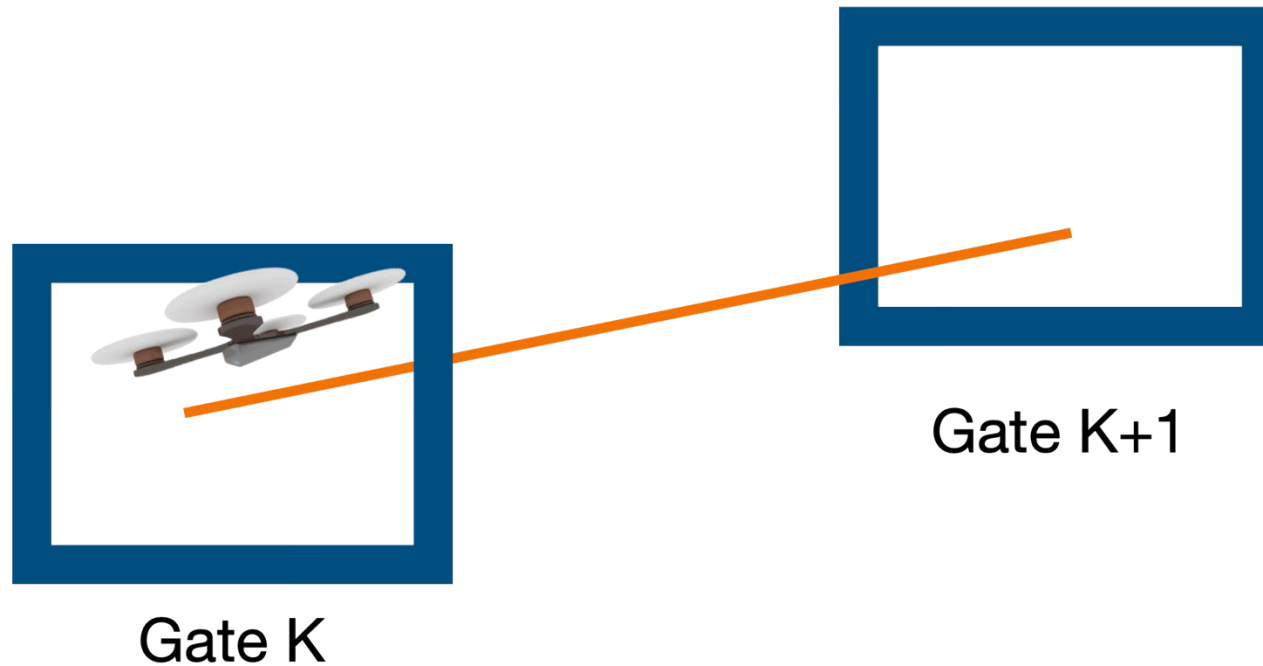
- We will define:
 - Rewards
 - Agent and Environment
 - The components of an RL Agent
 - Categories of RL agents
 - Exploration vs Exploitation
- A lot of examples along the way

Rewards

- A **reward** r_t is a scalar feedback signal.
- Indicates how well agent is doing at step t .
- The RL problem: maximize cumulative reward $\sum_t r_t$.
- Reinforcement learning is based on the **reward hypothesis**:
 - *All goals can be described by the maximization of expected cumulative reward.*

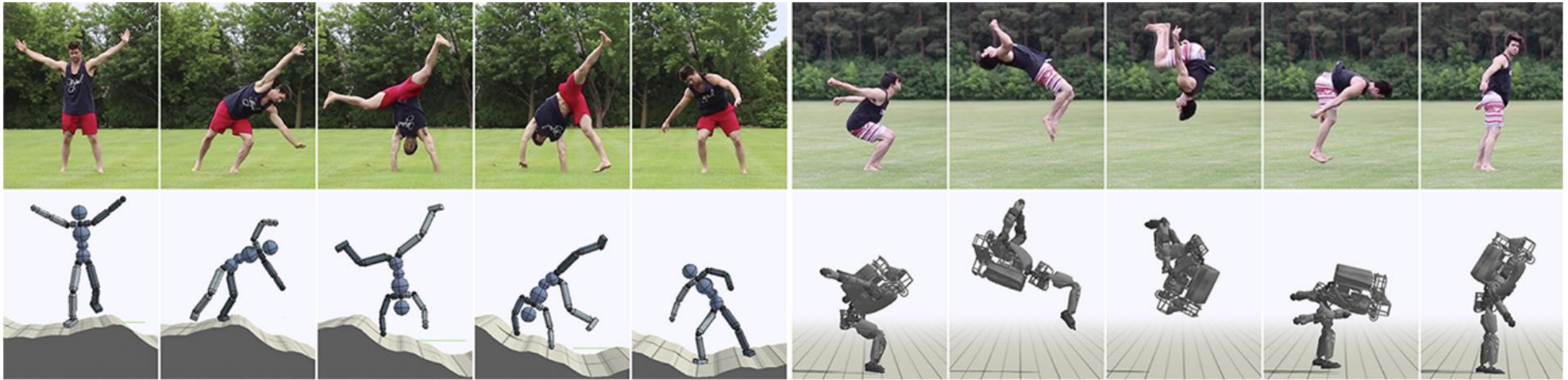
Dense Rewards

- At every time step, often non-zero.
- Rewards the agent for performing a certain behavior.



Dense Rewards

- At every time step, often non-zero.
- Rewards the agent for performing a certain behavior.

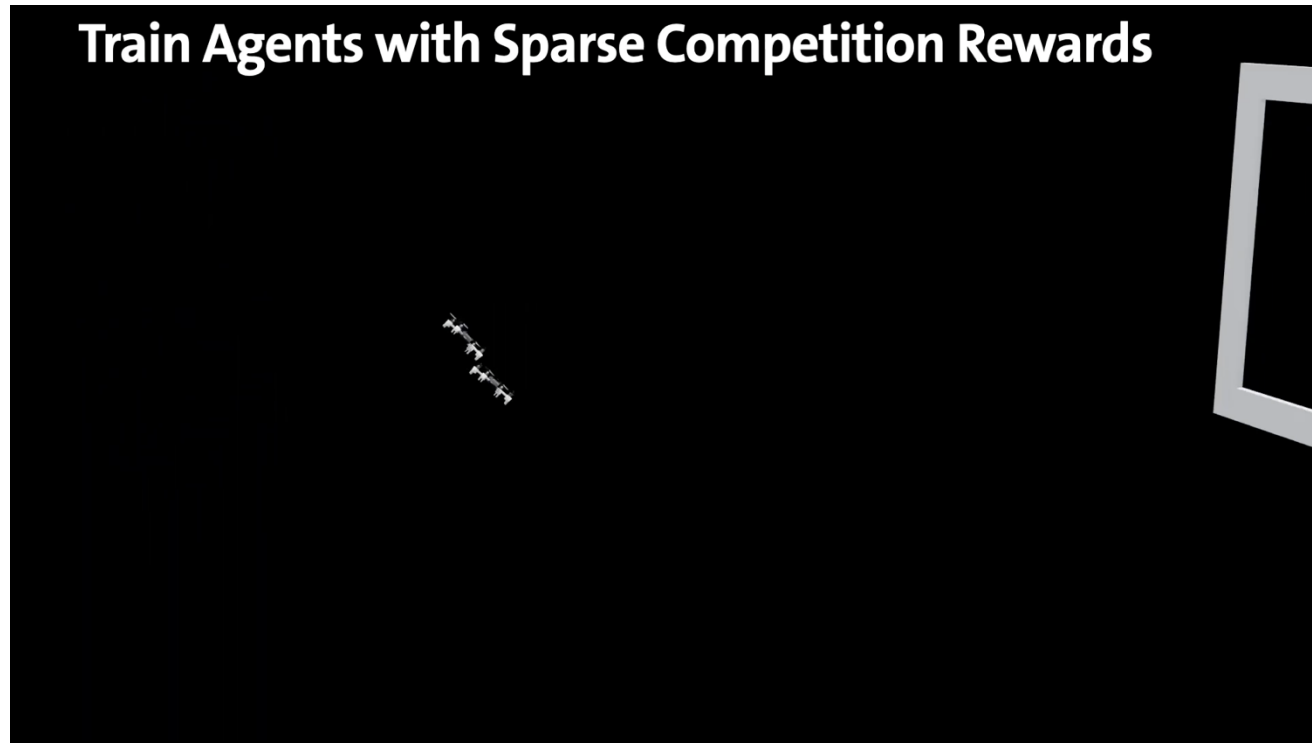


Dense Rewards

- At every time step, often non-zero.
- Rewards the agent for performing a certain behavior.
- What is the problem of dense rewards?

Sparse Rewards

- At every timestep, often zero. Non-zero only at key events.
- Rewards agent for achieving goals, not following behaviors.



Sparse Rewards

- Other examples:
 - Control a power station:
 - + bonus for decreasing power usage wrt baseline
 - - penalty for increasing power usage wrt baseline
 - Manage an investment portfolio:
 - + bonus for outperforming the market
 - - penalty for underperforming the market
- What are the disadvantages of this formulation?

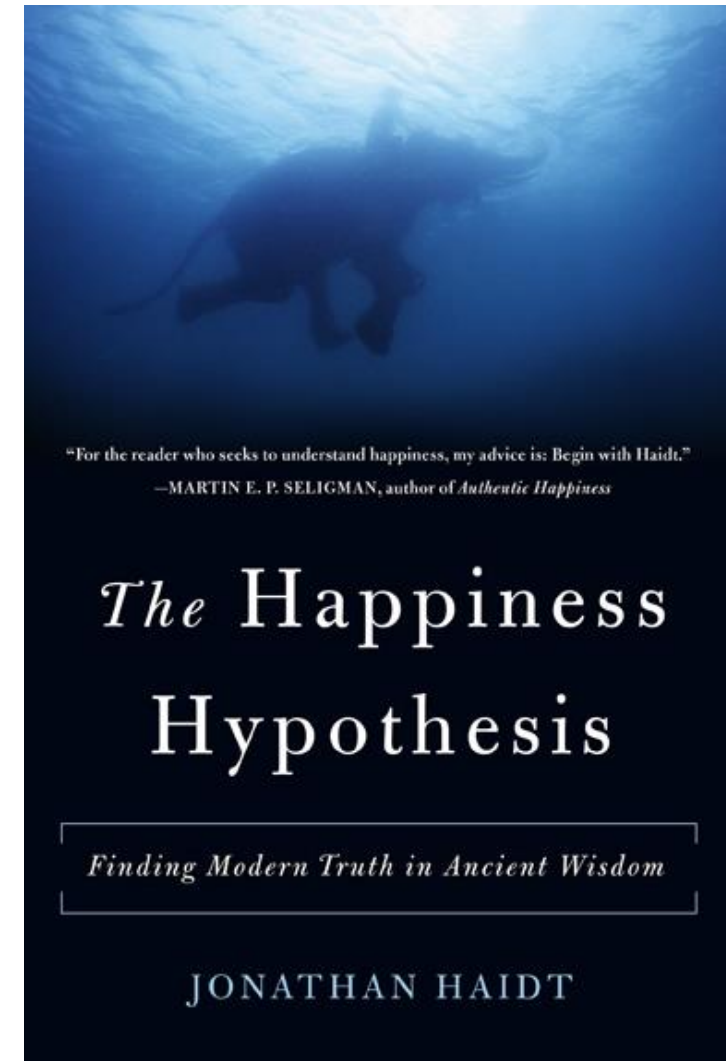
The core challenge of Sequential Decision Making

... and of living a happy life 😊

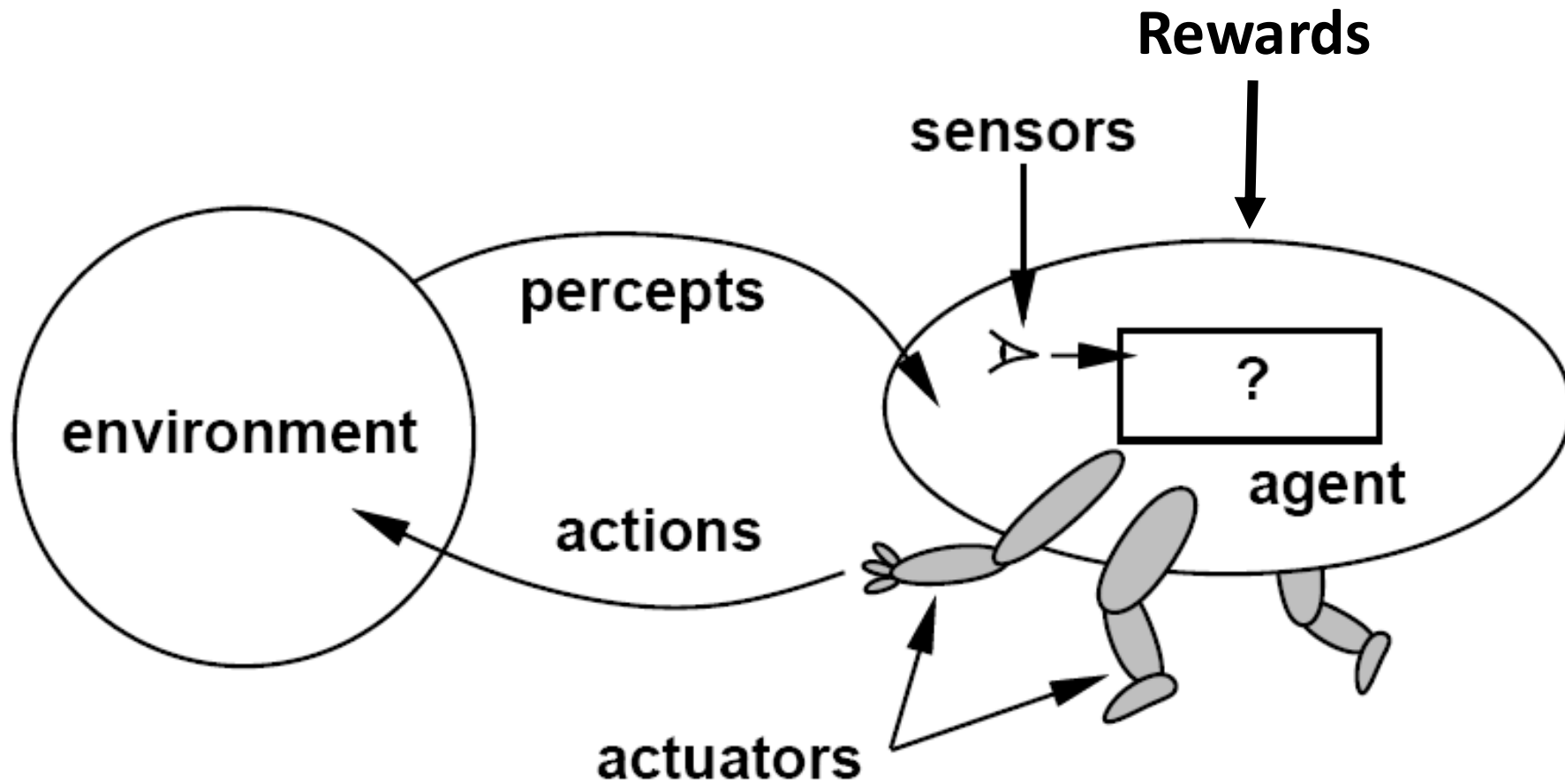
- Goal: select actions now to maximize total future reward.
- Actions may have long term consequences
- Rewards may be delayed
- Sometimes, it's better to sacrifice immediate reward to gain more long-term reward.
- Even us humans are often not very good at it.

Curious about decision-making in humans?

- The Happiness Hypothesis: Finding Modern Truth in Ancient Wisdom

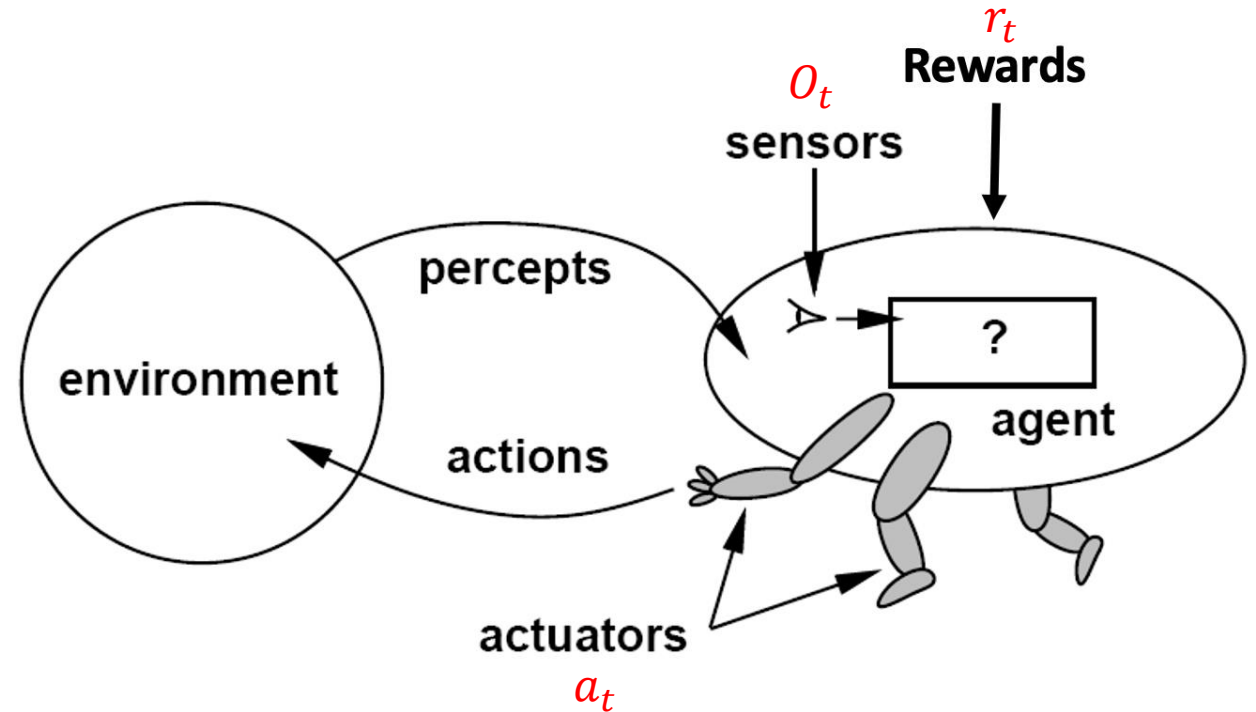


Agent and Environment



Agent and Environment

- At each step t the agent:
 - Generates observation o_t
 - Receives scalar reward r_t
 - Executes action a_t
- The environment:
 - Receives action a_t
 - Emits “causes” of observation o_{t+1}
- The reward:
 - From the agent or an external process, not from the environment.



History and State

- The history is the sequence of observations, actions, and rewards

$$h_t = o_1, r_1, a_1, \dots, a_{t-1}, o_{t-1}, r_{t-1}$$

- It contains all variables up to time t
 - Example: the entire sensorimotor stream of a robot.
- What happens next depend on h_t
- Formally, we define **state** as a function of history:
$$s_t = f(h_t)$$

The Environment State

- The **environment state** s_t^e is the environment's private representation
 - whatever data the environment uses to pick the next observation/reward
- The environment state is not usually visible to the agent.
- Even if s_t^e is visible, it may contain irrelevant information or be infinite dimensional.

The Agent State

- The **agent state** s_t^a is the agent's internal representation
$$s_t^a = f(h_t)$$
- Whatever information the agent uses to pick the next action
- It is the information used by reinforcement learning algorithms
- It can be as simple as the last observation o_t

The Markov State

- A state is Markov iff:

$$P[s_{t+1}|s_t] = P[s_{t+1}|s_1, s_2, \dots, s_t]$$

- Intuitive definition: *once the current state is known, you can forget about the past.*
- Formal definition: *the state is sufficient statistic of the future.*
 - The environment state s_t^e is Markov by definition.
 - The full agent history h_t is also a Markov state by definition.
 - The agent state s_t^a might not be Markov.

Example: Autonomous Driving

- What is a_t, o_t ?
- What is s_t^a ?
- What is s_t^e ?
- Is s_t^a a Markov State?

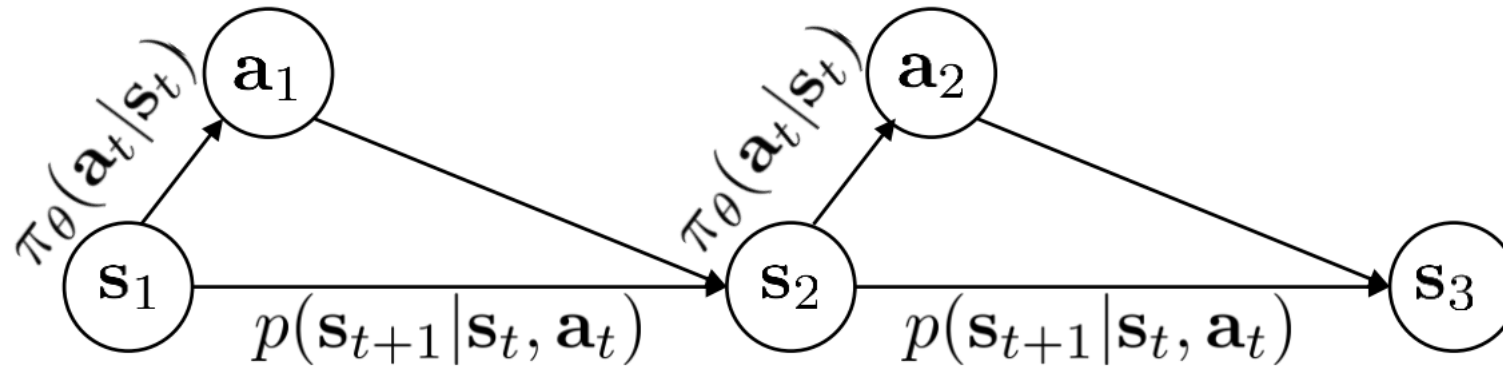


Fully Observable Environments

- **Full observability**: agent directly observes environment state.

$$o_t = s_t^a = s_t^e$$

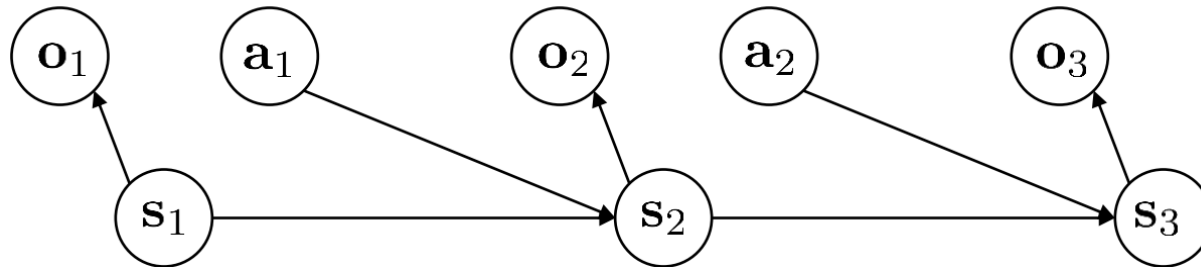
- Formally, this is a **Markov Decision Process** (MDP)



- Unrealistic for most real-world problems. But makes the math easier 😊

Partially Observable Environments

- Agent does not access the full environment state (car example).
- Formally, this is a **Partially Observable Markov Decision Process** (POMDP)



- Agent must construct its own state representation s_t^a . Examples:
 - Complete history h_t
 - Beliefs of environment state: $s_t^a = (P[s_t^e = s_1], \dots, P[s_t^e = s_n])$
 - Recurrent neural network: $s_t^a = NN(s_{t-1}^a, o_t)$

The Key Components of an RL Agent

- An RL agent may include any of following components:
 - **Policy** $\pi_{\theta}(a_t | s_t^a)$, the decision-making function with parameters θ
 - **Value function** $V(s_t^a)$ or **Q function** $Q(s_t^a, a_t)$ measure how “good” is each state and/or action.
 - **Model** $p(s_{t+1}^a | s_t^a, a_t)$ agent’s predictive representation of the environment.

Policy

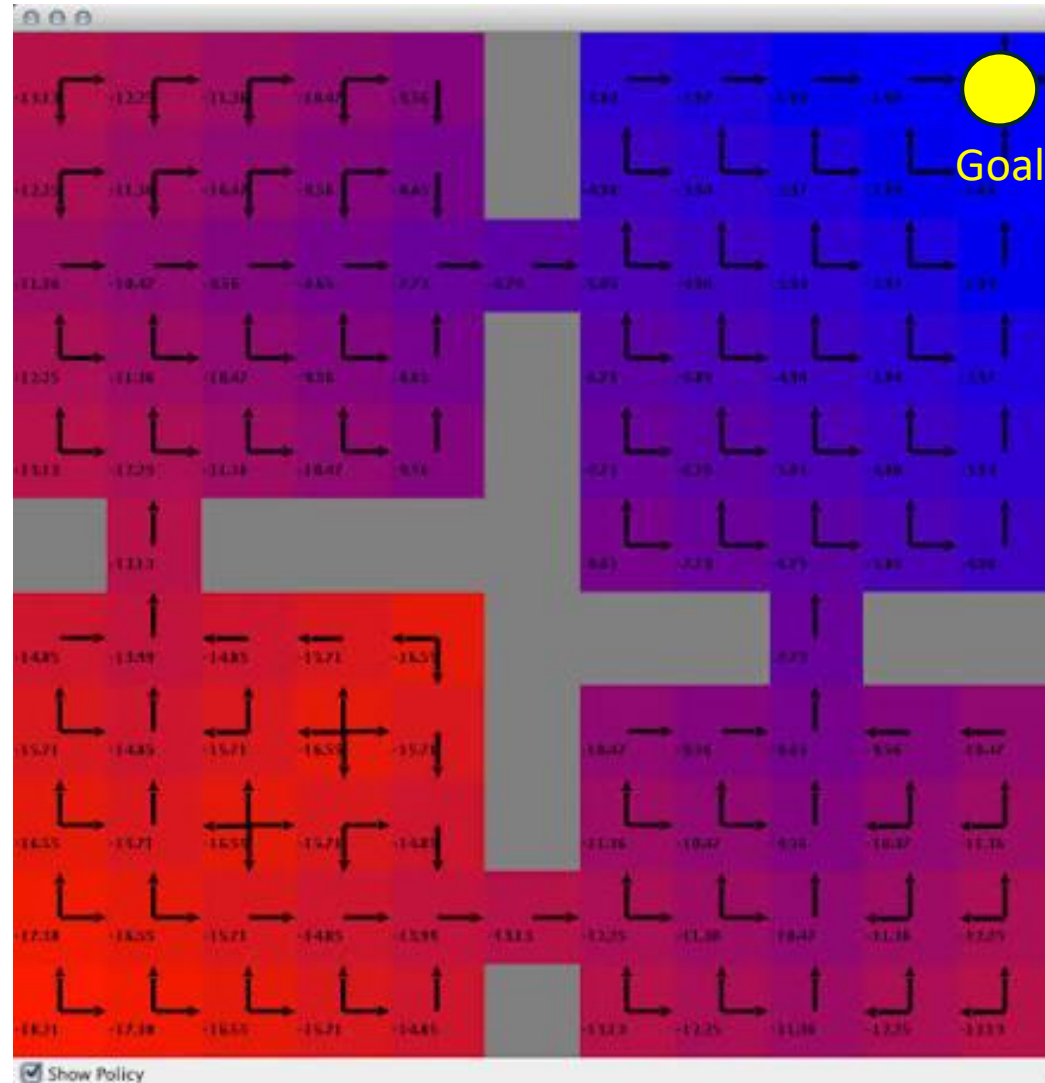
- A policy is the agent's behavior
- It is a parametric map from state to action
 - Deterministic policy. Example: $a = \pi_{\theta}(s) = \theta s$
 - Stochastic policy: $\pi(a | s) = \mathbb{P}[a_t = a | s_t = s]$.
- The RL problem consists in finding the parameters θ that maximize cumulative future rewards.

Value Function

- Value function is a prediction of future reward up to the “end”.
- Used to evaluate how good a state is
- Formal definition:

$$V(s) = \mathbb{E}[r_t + r_{t+1} + \dots + r_{t+N} \mid s_t = s]$$

Value Function Visualization



Q Function

- Q function is a prediction of future reward up to the “end”, taking action a at time t .
- Used to evaluate how good a state is when taking a specific action.
- Formal definition:

$$Q(s, a) = \mathbb{E}[r_t + r_{t+1} + \dots + r_{t+N} \mid s_t = s, a_t = a]$$

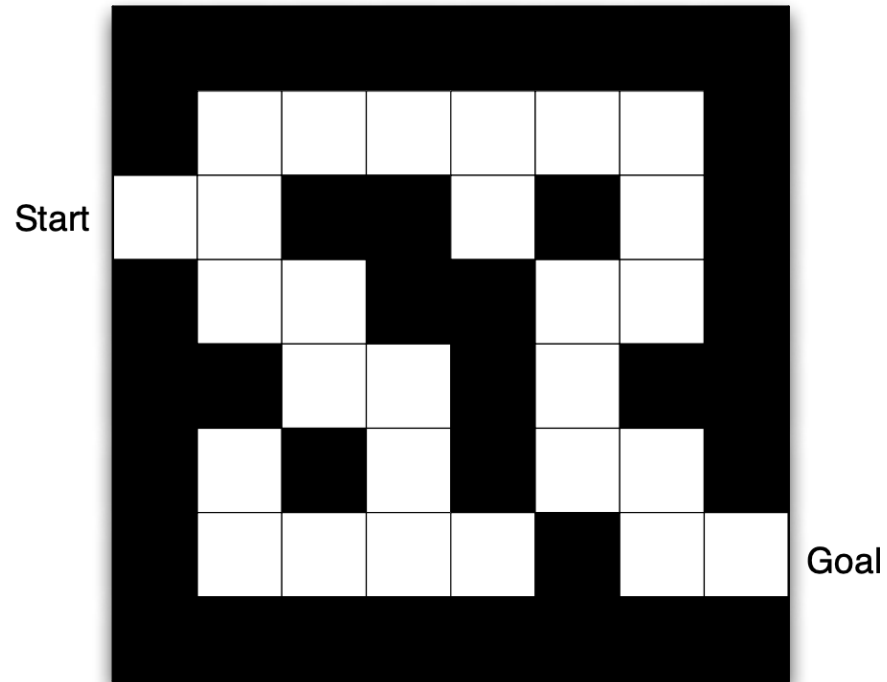
Model

- A **model** predicts what the environment will do next
- P predicts the next state
- (if rewards are stochastic) r predicts the next (immediate) reward

$$P_{ss'}^a = \mathbb{P}[s_{t+1} = s' | s_t = s, a_t = a]$$

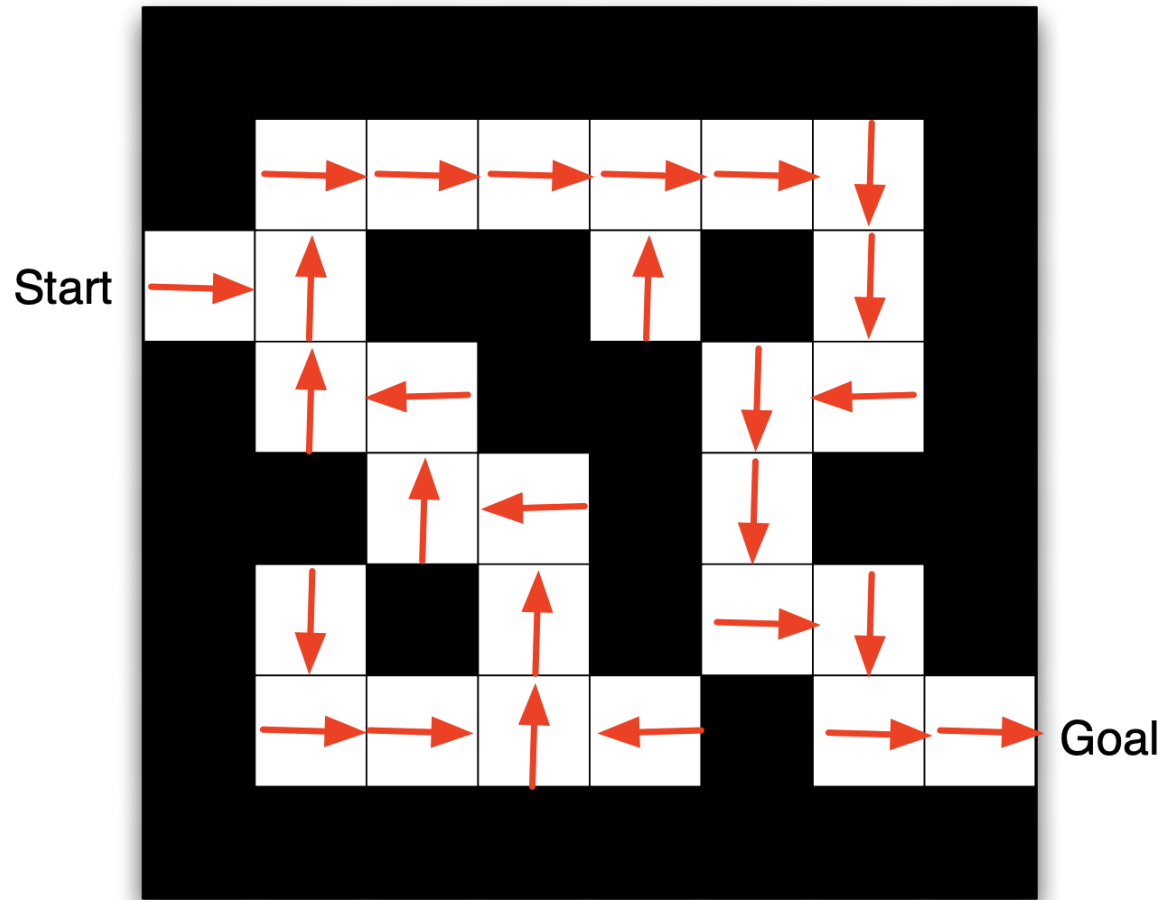
$$r(s, a) = \mathbb{E}[r_{t+1} | s_t = s, a_t = a]$$

Maze Example



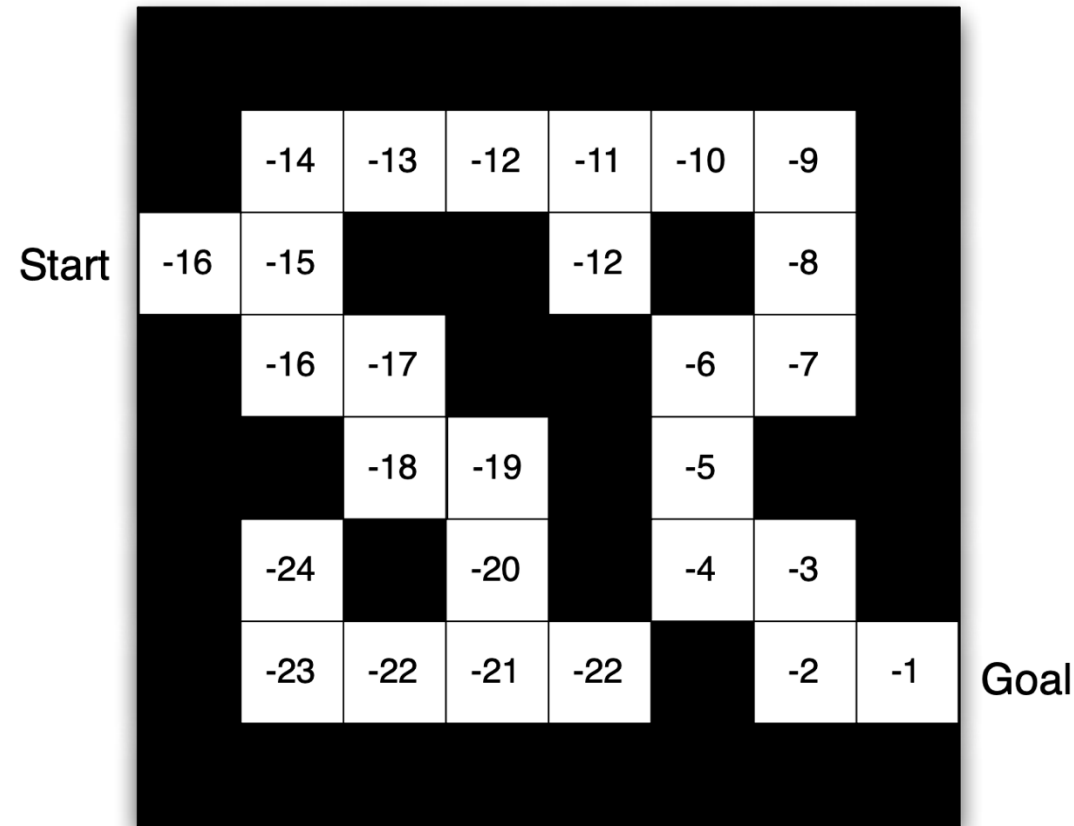
- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location

Maze Example: Policy



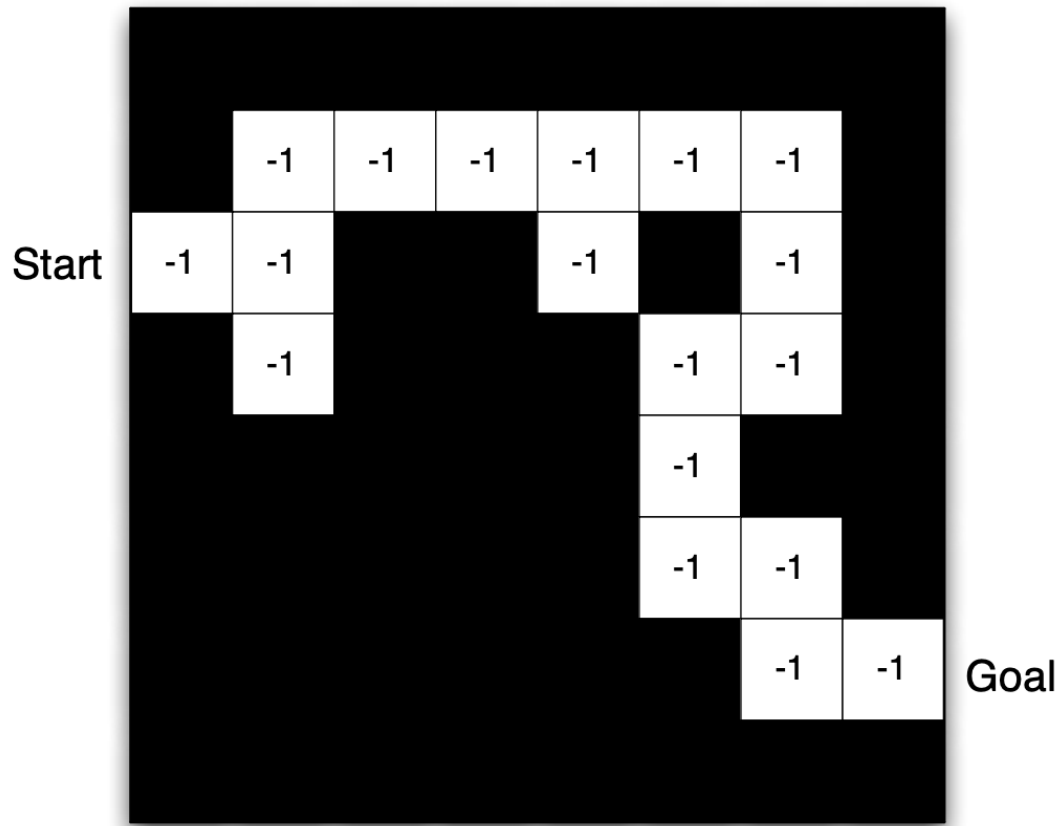
Arrows represents the policy for each state s .

Maze Example: Value Function



Numbers represent value of each state s

Maze Example: Model



- Agent may have an internal model of the environment
- Dynamics: how actions change the state
 - in the example, grid dynamics
- Rewards: how much reward for each state
 - In the example, always -1
- The model may be imperfect.

Categorizing RL agents

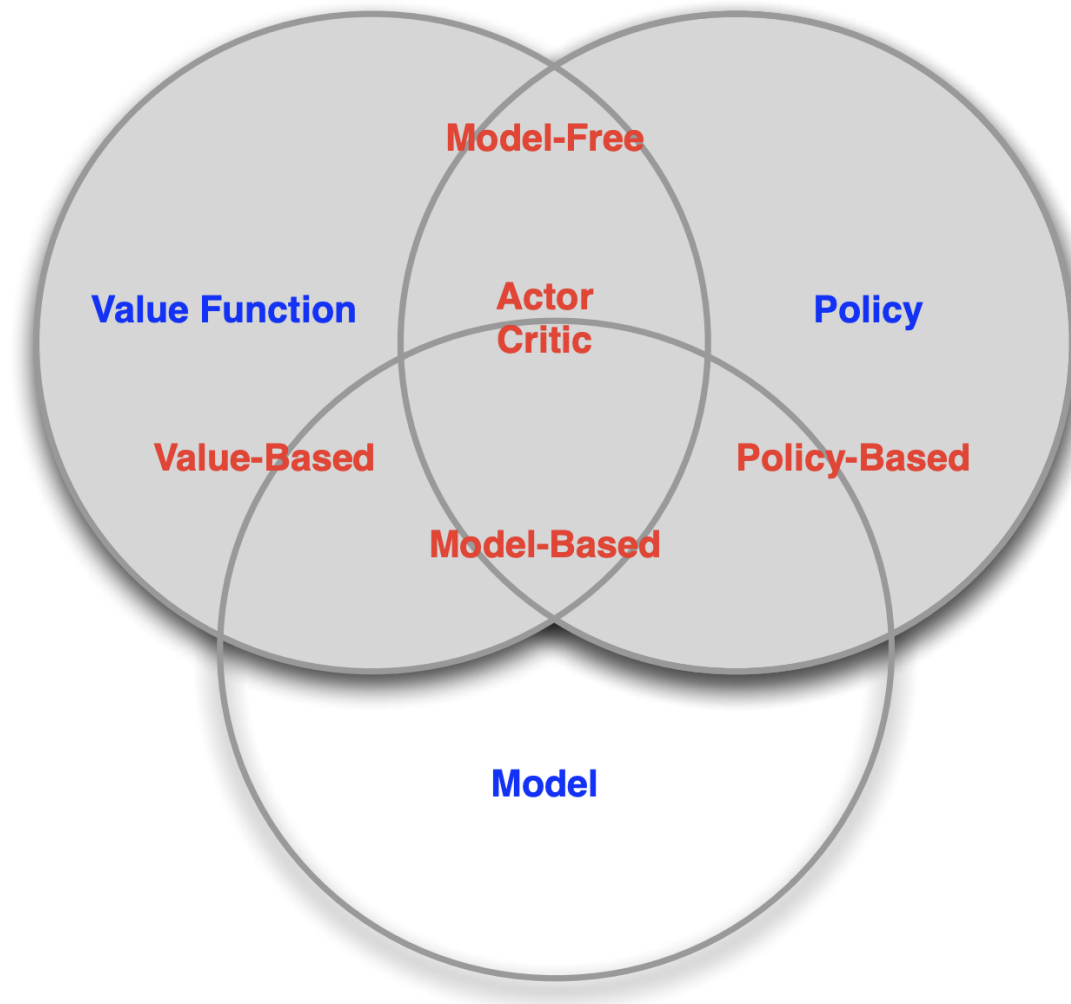
Axis 1: How to make decisions

- Value Based
 - No Policy (Implicit)
 - Value/Q Function
- Policy Based
 - Policy
 - No Value/Q Function
- Actor Critic
 - Policy
 - Value Function

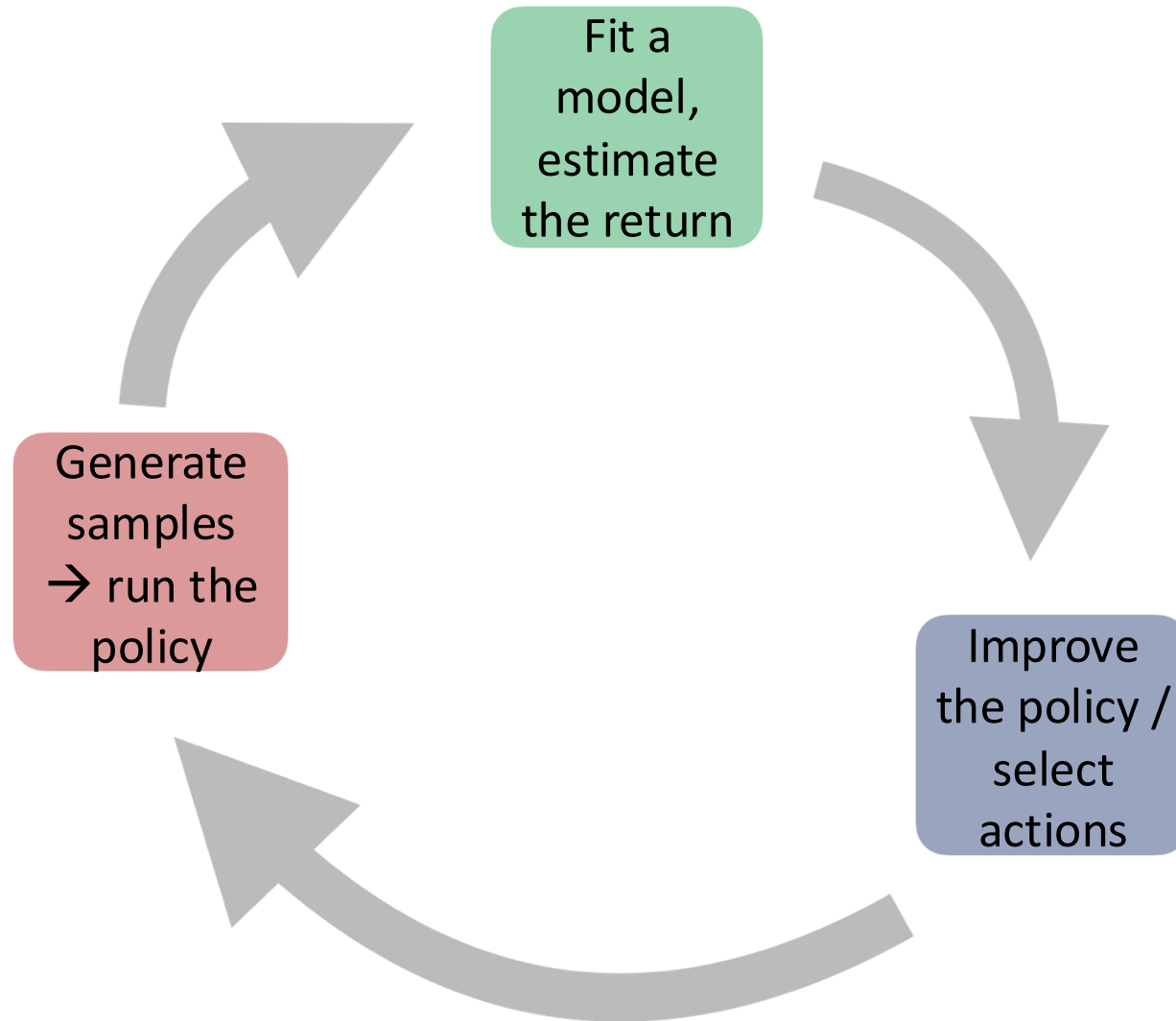
Axis 2: Environment Representation

- Model-based
 - Policy and/or Value/Q Function
 - Model
- Model-free
 - Policy and/or Value/Q Function
 - No Model

RL Agent Taxonomy

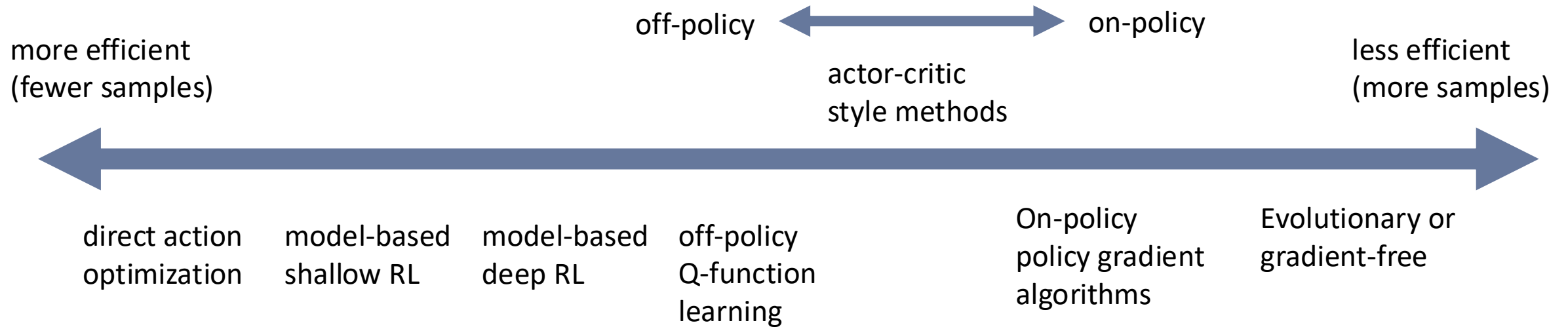


The general structure of an RL algorithm



Which part is the most expensive?

Categorizing RL Algorithms (Sample Efficiency)



Why not always use the most sample-efficient algorithm available?

Learning and Planning

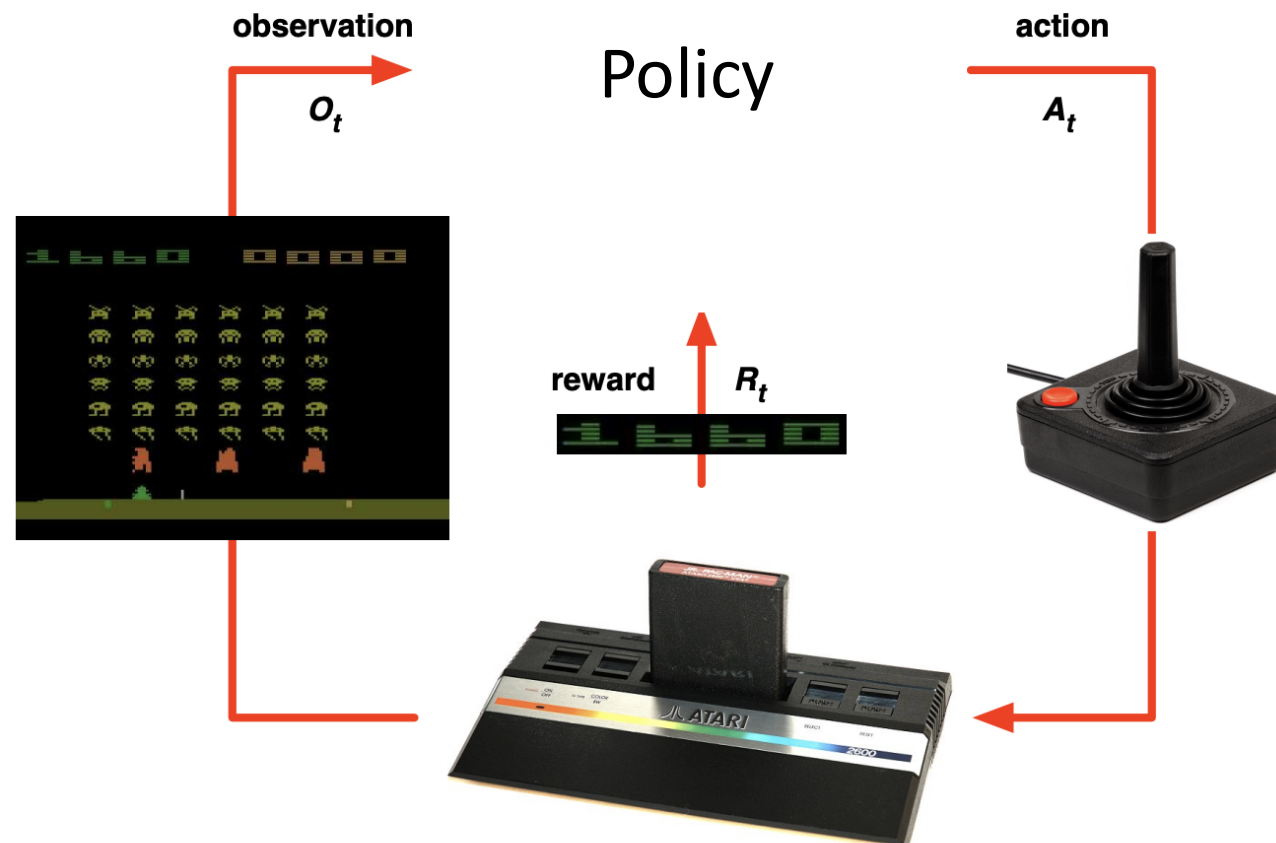
- Reinforcement Learning:

- The environment is initially unknown.
- The agent interacts with the environment.
- The agent improves the policy and potentially its model.

- Planning:

- A model of the environment is known.
- The agent does not interact with the environment to improve the policy, but only performs computation within its model.
- The output is a set of pre-computed actions to maximize the reward.

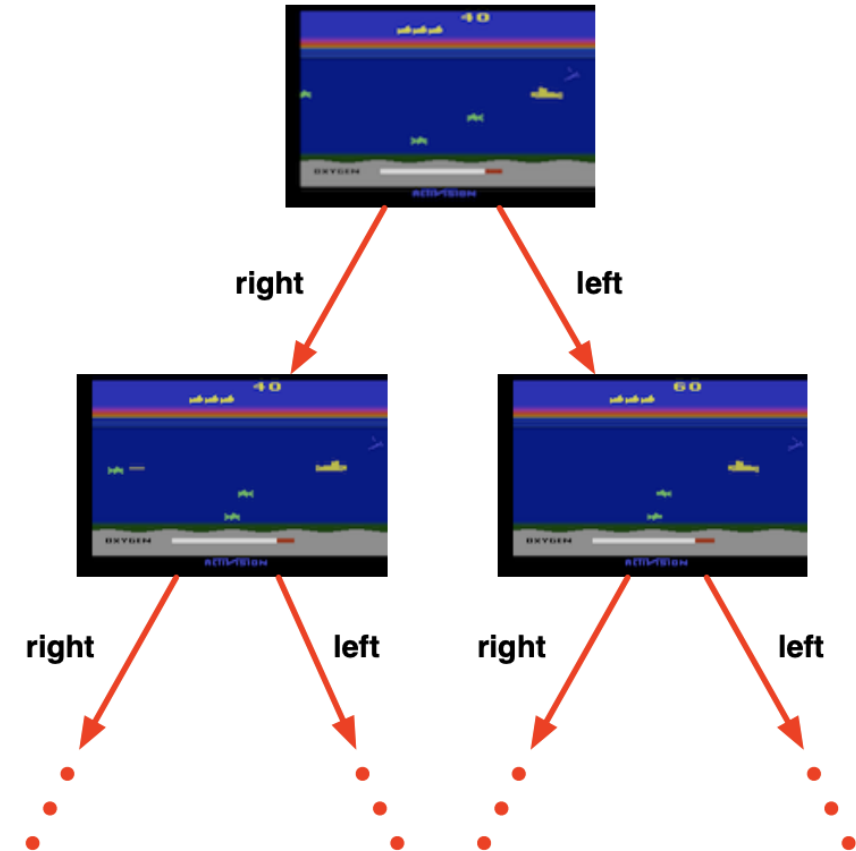
Atari Example: Reinforcement Learning



- Rules of the game are unknown
- Learn directly from interactive game-play
- Pick actions on joystick, see pixel scores

Atari Example: Planning

- Rules of the game are known
- Can query an emulator
 - Perfect model is available to the agent
- Plan ahead to find optimal policy
 - Using, for example, tree search



Exploration and Exploitation

- Reinforcement learning is like trial-and-error learning.
- The agent should discover a good policy from its experiences of the environment, without losing too much reward along the way.
- *Exploration* finds more information about the environment
- *Exploitation* exploits known information to maximize reward
- It is usually important to explore as well as exploit.
 - Time to try that new restaurant? 😊

Questions?